

TARUMT Resorts Assignment 保姆级教学与基础架构说明

这份 PDF 放在 assignment folder 外面。assignment folder 本身只保留 Java 代码骨架，方便你们直接拿去 NetBeans/Terminal 跑。

0. 现在这个版本是什么？

现在的目标不是生成最终作业答案，而是先搭一个干净、可分工、符合 ECB 思路的基础骨架。

- Terminal 主菜单已经有：Walk-In、VIP、Housekeeping、Front Desk、Loyalty。
- 每个 module 有自己的 Boundary UI 和 Control。
- ADT package 有 AVL / Queue / Stack / List 的接口和基础类。
- AVLTree 目前是 TODO scaffold，因为 AVL 是评分核心，应该由你们 team 自己完成。
- assignment/ 里面应该只保留 src/ 代码结构，不把教学 PDF 或说明文档塞进去。

1. 最终 folder 应该长什么样？

```
/Users/wilson/Documents/datastructure/
|-- Assignment_Baomu_Guide.pdf    <-- 这份教学 PDF，在外面
`-- assignment/
    |-- src/
        |-- main/
        |-- adt/
        |-- entity/
        |-- boundary/
        |-- control/
        |-- dao/
        |-- utility/
```

重点：assignment 是 project code folder，PDF 是学习资料，两个分开。这样交给组员时不会乱。

2. 为什么全部 Java code 都 under src？

Java project 通常会把源代码放在 src，因为 src 代表 source code。这样可以把源码、编译结果、文档、data 分清楚。

- 源码在 src/，编译后的 .class 放 build/，不会混在一起。
- package 名字和 folder 对应。例如 package adt; 的文件放 src/adt/。
- IDE 例如 NetBeans、IntelliJ、Eclipse 都默认认识 src 这种 source root。
- Terminal 编译时可以直接编译 src 底下所有 .java。
- 以后如果有 data/、docs/、reports/，它们不会被误当成 Java package。

present 时可以这样讲：

We put all Java source files under src because src is the source root.
Each subfolder matches a Java package, such as adt, entity, boundary and control.
Compiled class files go to build, so source code and output files are separated.

3. MacBook Terminal 运行方式

前提：电脑已经安装 JDK，并且 terminal 可以用 javac。先检查：

```
javac -version
java -version
```

进入 assignment folder：

```
cd /Users/wilson/Documents/datastructure/assignment
```

编译并运行：

```
rm -rf build
mkdir build
javac -d build $(find src -name "*.java")
java -cp build main.Main
```

如果看到 Unable to locate a Java Runtime，意思是 Mac 还没有配置 JDK。安装 JDK 后再跑，或直接用 NetBeans 配好的 JDK 跑。

4. Windows 运行方式

Windows 也要先安装 JDK，并确认 Command Prompt 或 PowerShell 可以用 javac：

```
javac -version
java -version
```

Command Prompt 版本：

```
cd /d C:\Users\<your-name>\Documents\datastructure\assignment
rmdir /s /q build
mkdir build
dir /s /b src\*.java > sources.txt
javac -d build @sources.txt
java -cp build main.Main
del sources.txt
```

PowerShell 版本：

```
cd C:\Users\<your-name>\Documents\datastructure\assignment
Remove-Item -Recurse -Force build -ErrorAction SilentlyContinue
New-Item -ItemType Directory build
$files = Get-ChildItem -Recurse src -Filter *.java | ForEach-Object { $_.FullName }
javac -d build $files
java -cp build main.Main
```

NetBeans 版本：Open Project 或新建 Java project 后，把 assignment/src 下的 package 放进 source root。Main class 选择 main.Main。

5. 整体架构：Console + ECB + ADT + DAO

User

- > Boundary UI
- > Control
- > ADT / Entity / DAO
- > Utility when needed

Layer	中文理解	里面放什么	不应该放什么
Boundary	terminal 画面层	menu, input, output, Scanner	业务逻辑、AVL root、file save
Control	业务大脑	流程判断、调用 ADT、创建 Entity	大量 println、底层 Node rotation
Entity	资料对象	Booking, Member, Room, getter/setter	Scanner、menu、AVL 操作
ADT	资料结构工具	AVL, Queue, Stack, List	hotel 业务规则
DAO	资料来源/保存	seed/load/save/repository	terminal UI
Utility	共用 helper	InputUtil, IdGenerator, DateUtil	module-specific business

一句话：Boundary 负责问用户，Control 负责决定做什么，ADT 负责怎么存，Entity 负责存什么，DAO 负责资料从哪里来。

6. 每个 package 要写什么？

Package	责任	现在/之后的重点
main	入口	Main.java 只调用 new MainMenuControl().run();
adt	主角数据结构	SearchTreeInterface, AVLTree, Queue, Stack, List。AVL 是 team main ADT。
entity	资料 class/object	Booking, Guest, Room, RoomStatus, Member, VipKey, PointsKey。
boundary	Terminal UI	MainMenuUI 和各 module UI，只做 input/output。
control	Module logic	各 module 的 run loop 和业务流程，例如 add/search/update/report。
dao	资料仓库	ResortRepository 共享同一批 ADT，DAO 之后负责 file/seed data。
utility	工具	InputUtil, MessageUI, IdGenerator, DateUtil, SortUtil。

7. AVL 到底存什么？Object 可以吗？

可以。AVL/BST 不是只能存 int。正确理解是：AVL 用 key 来比较大小，用 value 来存真正资料。

- key 必须可比较：Integer、String、或自己写 compareTo 的 class。
- value 可以是任何 object，例如 Booking、Member、RoomStatus。
- AVL 比的是 key，不是整个 object。

AVLTree<String, Member> memberById;

key = "M1001"

value = new Member("M1001", "Ali", 20, 3500)

如果你要 class 和 class 比，例如 VIP priority，就做一个 key class：

```
public class VipKey implements Comparable<VipKey> {
    private int tierRank;
    private long arrivalSeq;

    public int compareTo(VipKey other) {
        if (tierRank != other.tierRank) {
            return tierRank - other.tierRank;
        }
    }
}
```

```
    }  
    return Long.compare(other.arrivalSeq, arrivalSeq);  
  }  
}
```

present 讲法 : The AVL tree stores objects as values, but ordering is decided by comparable keys.

8. AVL 要提供给队员什么 method ?

Method	队员怎么用	Big-O
put(key, value)	add / update booking, member, room status	$O(\log n)$
get(key)	search confirmationNo/memberId/roomNo	$O(\log n)$
remove(key)	delete/cancel/remove record	$O(\log n)$
containsKey(key)	检查资料是否存在	$O(\log n)$
inOrder(visitor)	按照 key 从小到大 report	$O(n)$
reverseOrder(visitor)	VIP 从高到低 / points ranking	$O(n)$
rangeSearch(from, to, visitor)	日期/房号/分数范围 report	$O(\log n + k)$
maxEntry()	看最高 priority VIP	$O(\log n)$, 可用 max pointer 优化到 $O(1)$
removeMax()	拿走最高 priority VIP	$O(\log n)$

队员不需要知道 rotation 怎么写。他们只需要知道 control 层可以调用这些 ADT method。ADT 组才负责 Node、height、balance、rotation。

9. AVL rotation 最容易解释法 : LR / RL

核心口诀：先把弯的地方拉直，再对失衡 root 旋转。

LR 例子：insert 30, 10, 20。30 左边太高，但新节点插在 left child 的右边，所以是 Left-Right。

Before:

```

  30
 /
10
 \
 20

```

Step 1: left rotate at 10

```

  30
 /
20
 /
10

```

Step 2: right rotate at 30

```

  20
 / \
10 30

```

RL 例子：insert 10, 30, 20。10 右边太高，但新节点插在 right child 的左边，所以是 Right-Left。

Before:

```

  10
 \
  30
 /
 20

```

Step 1: right rotate at 30

```

  10
 \
  20

```

\
30

Step 2: left rotate at 10

20
/ \
10 30

为什么突然拿 10 或 30 来旋转？因为第一步不是在最上面的失衡点旋转，而是在失衡点的 child 上先修弯。LR 修 left child，RL 修 right child。

10. 每个 module 选什么辅助 ADT ?

Module	主 AVL	辅助 ADT	为什么
Walk-In Booking	bookingTree: confirmationNo -> Booking	Queue + List	Queue 做 FIFO 排队；List 做 report 暂存。
VIP Allocation	vipTree: VipKey -> Booking	maxEntry/removeMax	AVL 天然有序，最高 priority 在 max side。
Housekeeping	roomStatusTree: roomNo -> RoomStatus	Stack + List	Stack 做 undo/redo；List 做 status report。
Front Desk	bookingTree / roomTree	List, Hash 可选	AVL 查找 $O(\log n)$ 已够；Hash 可把 exact search 加到 average $O(1)$ ，但同步更麻烦。
Loyalty	memberById + memberByPoints	List	一个 AVL 查 memberId，一个 AVL 按 points 排名；List 做筛选 report。

重点：不是每个地方都硬塞 Hash。辅助 ADT 要解决真实问题：FIFO 用 Queue，undo 用 Stack，排序/ranking 用 AVL traversal，临时 report 用 List。

11. Walk-In / Standard Booking 怎么做？

Add booking:

1. Boundary 问 guest details
2. Control create Booking
3. bookingTree.put(confirmationNo, booking)
4. walkInQueue.enqueue(booking)

Allocate next:

1. booking = walkInQueue.dequeue()
2. 如果 booking cancelled，就 skip
3. assign room
4. bookingTree.put(confirmationNo, updatedBooking)

解释：Queue 保证 first come first served；AVL 保证之后可以用 confirmation number 搜索。

12. VIP Priority Allocation 怎么做？

VipKey = tierRank + arrivalSeq + confirmationNo

Add VIP:

vipTree.put(vipKey, booking)

View highest:

vipTree.maxEntry()

Allocate highest:

Entry<VipKey, Booking> next = vipTree.removeMax()

解释：VIP priority 不是单纯用 booking object 比，而是做一个 VipKey。tier 越高，key 越大；同 tier 再用 arrivalSeq 处理先来先服务。

13. Housekeeping Undo/Redo 为什么 AVL 不能单独做？

AVL 的功能是 search/insert/delete/sorted traversal。它不天然记得以前发生过什么。Undo 需要 history，history 的天然 ADT 是 Stack。

- AVL 只保存 current state：room 101 现在是 Dirty/Ready/Inspected。
- Stack 保存 previous states：上一次、上上次、再上一次。
- Undo 是 pop previous；Redo 是 pop next。

Update room status:

```
old = roomStatusTree.get(roomNo)
undoStack.push(copy(old))
redoStack.clear()
roomStatusTree.put(roomNo, newStatus)
```

Undo:

```
current = roomStatusTree.get(roomNo)
redoStack.push(copy(current))
previous = undoStack.pop()
roomStatusTree.put(roomNo, previous)
```

Redo:

```
current = roomStatusTree.get(roomNo)
undoStack.push(copy(current))
next = redoStack.pop()
roomStatusTree.put(roomNo, next)
```

设计上 Stack 不放进 RoomStatus entity，放在 Repository/Control 侧，因为 Entity 应该只代表资料，不应该知道 ADT history。

14. Front Desk 怎么做？

Search booking:

```
Booking b = bookingTree.get(confirmationNo)
```

Check room:

```
Room room = roomTree.get(roomNo)
RoomStatus status = roomStatusTree.get(roomNo)
```

Report:

```
bookingTree.inOrder(visitor)
```

filter result into List

HashTable 可选：如果老师问 exact search $O(\log n)$ 可不可以更快，可以说可以加 HashTable key -> Booking 到 average $O(1)$ ，但每次 AVL update 也要同步 Hash。基础版先不加，避免复杂度爆炸。

15. Loyalty 怎么做？

memberById:

```
key = memberId
value = Member
```

memberByPoints:

```
key = PointsKey(points, memberId)
value = Member
```

Update points:

1. oldMember = memberById.get(memberId)
2. memberByPoints.remove(old PointsKey)
3. oldMember.setPoints(newPoints)
4. memberById.put(memberId, oldMember)
5. memberByPoints.put(new PointsKey, oldMember)

解释：同一个 Member 可以被两个 AVL index 看到，一个用来 exact search，一个用来 ranking/report。

16. DAO 和 Repository 里面存什么？

DAO 不是 business logic，也不是 UI。DAO 是资料来源。当前基础版最重要的是 ResortRepository，因为多个 module 要共享同一批树。

```
ResortRepository
|-- bookingTree
|-- walkInQueue
|-- vipTree
|-- roomStatusTree
|-- roomUndoHistory
|-- roomRedoHistory
|-- memberById
|-- memberByPoints
`-- roomTree
```

为什么要 repository？因为 Walk-In 加的 booking，Front Desk 也要查。如果每个 Control 自己 new AVLTree，资料会分裂，module 之间看不到彼此的数据。

DAO 类之后可以做 file load/save，例如 BookingDAO.loadBookings()、MemberDAO.saveMembers()。但基础架构阶段先保留 skeleton。

17. 两个人开发 ADT 要怎么分？

Person	负责内容	产出
A: AVL core	Node, height, get, put, rotate, rebalance	AVLTree basic search/insert 可用
B: AVL advanced + helpers	remove, traversal, rangeSearch, maxEntry/removeMax, Queue/Stack/List check	module 可以调用完整 ADT method

模块同学只接 method，不碰 AVL 内部。比如 Walk-In 同学只需要 bookingTree.put/get 和 walkInQueue.enqueue/dequeue。

18. Presentation 最短讲法

Our project uses AVL Tree as the main ADT because it keeps records sorted and guarantees $O(\log n)$ search, insert and delete.

We store objects as values, but compare records using keys.
For example, confirmation number is the key and Booking is the value.

We also use auxiliary ADTs only when they naturally match the operation:

Queue for FIFO walk-in bookings,
Stack for housekeeping undo/redo,
List for temporary report results.

The project follows ECB:
Boundary handles terminal UI,
Control handles module logic,
Entity stores data,
ADT implements data structures,
DAO handles shared data and future file storage.

19. 现在你们下一步应该做什么？

- 先确认 JDK/NetBeans 可以跑 main.Main。
- ADT 组先完成 AVL get/put/rotation/traversal。
- Module 组先把 UI input 和 Control flow 写好，但先用 TODO method 接口。
- 等 AVL 完成后，把 module 接上 repository 里的 tree/queue/stack。
- 最后才做 file save/load、report polish、demo script。